

Pandas 입문

read_csv · 구조 파악

read_csv가 안 된다는 분의 90%는 경로 문제입니다 — 그래서 점검 루틴을 먼저 배웁니다

12_01 Pandas 입문과 데이터 불러오기 · 12_02 DataFrame 구조 PART 01

핵심 키워드

```
import pandas as pd · Series · DataFrame · 인덱스 · CSV  
pd.read_csv() · encoding · sep · index_col · nrows · usecols · FileNotFoundError  
.head() · .tail() · NaN · .shape · .columns · .index · .dtypes · .info()
```

2026년 8월 12일 (수) · 17일차 · 42~43차시

국립금오공과대학교 컴퓨터공학부 박민호

이 책을 읽는 법

뱃지 · 녹음 공백 구간 고지 · 앞뒤 회차와의 연결

1. 항목 뱃지

오늘은 **함수·메서드·속성**이 골고루 나옵니다. `pd.read_csv()`는 함수, `.head()`는 메서드, `.shape`은 속성입니다. 15일차부터 써 온 3초 판별법이 그대로 적용됩니다.

뱃지	판별법	이번 회차 예
함수	앞에 점(.)이 없습니다 — 모듈 이름이 앞에	<code>pd.read_csv()</code> · <code>pd.DataFrame()</code>
메서드	점 + 이름 + 소괄호 ()	<code>.head()</code> · <code>.tail()</code> · <code>.info()</code> · <code>.describe()</code>
속성	점 + 이름, 괄호 없음	<code>.shape</code> · <code>.columns</code> · <code>.index</code> · <code>.dtypes</code>
키워드	함수 괄호 안의 이름=값 — 키워드 인자	<code>encoding=</code> · <code>sep=</code> · <code>usecols=</code>
자료형	값의 종류	<code>Series</code> · <code>DataFrame</code> · <code>int64</code> · <code>object</code>
개념	문법이 아닌 배경 지식	CSV · 인코딩 · 상대경로 · NaN

2. 녹음 공백 구간 — 무엇이 비었고 어떻게 메웠나

이 회차는 전사본이 두 조각으로 나뉘어 있고 **중간 한 시간이 비어 있습니다**. 어떤 내용이 그 구간에 들어가는지 특정할 수 있으므로 교안과 코드로 복원했지만, 그 시간대의 **강사 구두 설명은 이 책에 없습니다**.

시간대	녹음	들어간 내용	이 책에서
~14:00	있음 · 151분	12_01 전체 — Pandas 입문 · <code>read_csv</code> · 옵션 · 실습 1~3	STEP 1~6
14:00~15:00	없음	12_01 실습 4~6 마무리 + 12_02 <code>head</code> · <code>tail</code> ·NaN · 실습 1·2	STEP 7 — 교안·코드로 복원
15:00~15:45	있음 · 45분	12_02 <code>shape</code> → <code>columns</code> → <code>index</code> → <code>dtypes</code> → <code>info</code> → <code>describe</code> 소개	STEP 8~9
15:45~	수업 종료	강사가 "오늘 준비한 자료는 여기까지"로 마무리 — 이후 실습 시간	해당 없음

주의

공백 구간을 특정할 수 있는 근거. 뒤쪽 녹음이 shape 설명으로 시작하는데, 교안 12_02의 순서는 head(p7) → tail(p9) → 행 개수 조절(p11) → NaN(p13) → 실습 1·2(p15·17) → shape(p19) 이다. 즉 shape 앞의 그 구간이 통째로 공백에 들어간다. 해당 내용은 교안 12_02 p5~p18, 강사 코드 12_02_01_head, 그리고 직접 본 Practice/12_02_example.py 실습 1·2로 덮었다.

개념

빠진 자료 하나. 강사 코드 12_02_practice_42_260812_2.py가 data/12_metro_small_2.csv를 참조하는데 이 파일이 폴더에 없습니다. 다만 실습 1은 "shape 찍어 보기" 수준이고 12_metro_small.csv(30행 7열)로 대체 가능하므로 진행에 지장이 없습니다.

3. 앞뒤 회차와의 연결

이 회차는 **넘파이(16일차)**와 **데이터프레임 선택(18일차)** 사이의 다리입니다. 강사가 뒤쪽 녹음에서 shape을 설명하며 한 말이 정확합니다 — "넘파이에서 얘기했던 shape하고 똑같죠?"

16일차 넘파이	17일차 Pandas (오늘)	18일차에서 이어짐
<code>np.loadtxt(...)</code>	<code>pd.read_csv(...)</code>	같은 파일을 더 편하게
<code>arr.shape</code> (행, 열)	<code>df.shape</code> 동일	그대로 사용
<code>arr.dtype</code> 전체 하나	<code>df.dtypes</code> 열마다	<code>object</code> 열 주의
숫자만 담김	시각·문자·숫자 혼합	품질등급 같은 정답 열
<code>data[:, 0]</code> 번호로	(다음 회차)	<code>df['오일온도']</code> 이름으로
<code>arr.mean()</code> 각각	<code>df.describe()</code> 8개 한 번에	18일차에서 본격

CONTENTS

목차

오늘 배운 것 전체 지도

장 · STEP	무엇을 배우나	핵심
CHAPTER 0	오늘의 지도 — Pandas 4단계 작업 흐름과 용어	불러오기→구조→ 미리보기→통계
STEP 1	왜 엑셀이 아니라 코드인가	하루 86,400줄
STEP 2	Series와 DataFrame — 두 개의 그릇	Series 한 열 · DataFrame 표
STEP 3	CSV와 read_csv 기본	<code>df = pd.read_csv(...)</code>
STEP 4	경로 — 안 된다는 분의 90%	<code>FileNotFoundError</code>
STEP 5	read_csv 옵션 다섯 가지	<code>encoding·sep·index _col·nrows·usecols</code>
STEP 6	불러오기 점검 루틴	경로→인코딩→구분자→ 확인
STEP 7	미리보기 — head와 tail, 그리고 NaN	<code>.head(n) · .tail(n)</code>
STEP 8	구조 파악 4종	<code>.shape .columns .i ndex .dtypes</code>
STEP 9	info와 describe — 첫 탐색 4단계 완성	<code>.info()</code> · Non-Null Count
부록 A	치트시트 — 오늘 나온 전부 요약	한 장 요약
부록 B	오류·함정 사전 + 제출 코드 교정	$\times \rightarrow \checkmark$
부록 C	실습 1~6 하이라이트 · 보충 · 다음 시간 예고	describe·행열 선택

오늘 하루의 흐름 — 문 열고 들어가서 방을 둘러보기

교안 p14가 오늘 전체를 **집을 살펴보는 일**에 비유합니다. 문을 열고 들어가고(불러오기), 방이 몇 개인지 둘러보고(구조 확인), 방 안을 들여다보고(미리보기), 살림살이를 셈해 본다(통계 요약). 베테랑 분석가도 새 데이터를 만나면 이 순서를 밟습니다.

① 불러오기
read_csv



② 구조 확인
shape · info



③ 미리보기
head · tail



④ 통계 요약
describe

설비 적용

오늘 다루는 데이터는 **지하철 공기압축기(MetroPT-3)** 로그입니다. 전동차의 브레이크와 출입문에 쓰는 압축공기를 만드는 설비로, 압축/배출/저장 압력·오일온도·모터전류·가동상태를 기록합니다. **시각(문자) · 숫자 · 상태(문자)가 한 표에 섞여 있는 것**이 핵심입니다 — 넘파이 배열로는 담을 수 없는 형태이고, 그래서 Pandas가 필요합니다.

오늘의 지도

Pandas 4단계 작업 흐름 · 오늘 나온 이름 전체

0-1. Pandas 데이터 분석 4단계

오늘 배우는 도구들은 낱개로 외우면 금방 뒤섞입니다. **네 단계 중 어디에 쓰는 것인지**로 묶어 두면 훨씬 오래 갑니다.

단계	하는 일	도구	비유
STEP 1 불러오기	CSV 파일을 DataFrame으로 변환	<code>pd.read_csv()</code>	문 열고 들어가기
STEP 2 구조 확인	행·열 수, 자료형 같은 뼈대 확인	<code>.shape · .columns · .info()</code>	방이 몇 개인지 둘러보기
STEP 3 미리보기	실제 값을 직접 눈으로 확인	<code>.head() · .tail()</code>	방 들여다보기
STEP 4 통계 요약	평균·최대·최소로 첫 신호 잡기	<code>.describe()</code>	살림살이 셈하기

개념

강사가 뒤쪽 녹음 마지막에 정리한 문장이 이 회차의 결론입니다 — "read_csv 이후에는 head · shape · info · describe 이 네 가지를 기본적으로 실행해 주는 게 권장된다." 순서가 조금 다르지만 같은 이야기입니다. 이 네 줄이 모든 데이터 분석의 출발점입니다.

0-2. 오늘 나온 이름 전체 — 분류부터 해 두고 시작

분류	이름	한 줄
키워드	<code>import pandas as pd</code>	전 세계 공통 별명 — 모든 분석의 첫 줄
함수	<code>pd.read_csv(경로, ...)</code>	CSV를 DataFrame으로 읽기
자료형	Series	1차원 — 표의 한 열
자료형	DataFrame	2차원 표 — 분석의 중심 객체
개념	인덱스 (index)	각 행에 붙은 번호표 — 0부터
개념	CSV	쉼표로 값을 구분한 텍스트 파일
개념	상대경로 vs 절대경로	보통 상대경로 · 슬래시(/) 사용

분류	이름	한 줄
개념	FileNotFoundError	경로 문제 — 오류의 90%
키워드	encoding=	한글 깨짐 — utf-8 계열 먼저, 안 되면 cp949
키워드	sep=	구분자 — 칸이 뭉치면 이것
키워드	index_col=	특정 열을 행 이름표로
키워드	nrows=	위에서 몇 줄만 읽기
키워드	usecols=	필요한 열만 골라 읽기
메서드	.head(n) · .tail(n)	앞·뒤 n줄 미리보기
개념	NaN	Not a Number — 결측치 표시, 0과 다름
속성	.shape	(행, 열) 튜플
속성	.columns	열 이름 목록
속성	.index	행 이름표 범위
속성	.dtypes	열별 자료형
메서드	.info()	구조 + 결측 종합 — Non-Null Count
메서드	.describe()	숫자 열 8개 통계 (다음 회차 본격)

STEP 1

왜 엑셀이 아니라 코드인가

설비 데이터의 규모가 곧 Pandas를 배우는 이유

강사가 첫 질문으로 던진 것이 이것입니다 — "표 데이터를 코드로 다뤄야 하는 이유가 뭘까요? 엑셀로도 되잖아요."
답은 규모에 있습니다.

1-1. 하루 86,400줄

공장 설비 한 대가 1초에 한 번 측정하면 하루 **86,400줄**입니다. 센서가 수십 개면 하루 수백만 줄이 쌓입니다. 엑셀로 스크롤하며 눈으로 볼 수 있는 양이 아니다.

숫자로 보는 이유

```
# 계산해 보면 감이 온다
초당_1회 = 60 * 60 * 24
print(f"설비 1대 하루: {초당_1회:,}줄")
print(f"센서 20개 : {초당_1회 * 20:,}줄")
print(f"30일 누적 : {초당_1회 * 20 * 30:,}줄")
```

```
# 엑셀 한 시트 최대 행 수
print(f"엑셀 시트 한계: {1048576:,}줄")
```

- ▶ 설비 1대 하루: 86,400줄
- ▶ 센서 20개 : 1,728,000줄
- ▶ 30일 누적 : 51,840,000줄
- ▶ 엑셀 시트 한계: 1,048,576줄

센서 20개짜리 설비 하루치가 이미 엑셀 한 시트 한계(104만 행)를 넘습니다. **한 달치는 시트 50장이 필요합니다.** 이것이 Pandas를 배우는 이유입니다.

1-2. 코드로 다루는 세 가지 이점

이점	무엇이 좋은가	설비 분석에서
① 대량 처리	수백만 줄도 명령 한 줄로	온도 평균·100도 초과 시점 추출이 클릭 없이 끝
② 재사용	한 번 짰 코드를 새 데이터에 그대로	다음 달 데이터가 와도 다시 돌리기만
③ 정확성	같은 작업을 오차 없이 반복	고장 신호 하나가 큰 사고로 이어지는 분야

1-3. 엑셀과 Pandas는 경쟁자가 아니다

교안이 분명히 합니다 — **역할이 다른 도구**입니다. 실무에서는 둘을 함께 씁니다. 무거운 처리는 Pandas로 하고, 결과를 보기 좋게 정리하는 것은 엑셀로 합니다.

상황	적합한 도구	이유
수백 줄을 눈으로 보며 색칠	엑셀	적은 양을 섬세하게 다루는 칼·도마
수십만 줄 이상, 반복 작업	Pandas	재료가 많고 같은 작업이 반복될수록 진가
한 시트 104만 행 초과	Pandas	엑셀은 아예 열리지 않음
결과를 팀에 공유	Pandas → 엑셀	처리는 코드로, 전달은 엑셀 파일로

연결

강사가 뒤쪽 녹음에서 **SQL**을 언급했습니다. 데이터베이스에서 쿼리문으로 데이터를 조작해 통계를 내는 접근법인데, Pandas와는 방식이 다르지만 **둘을 겸해 쓰면 더 효율적**이라고 알려져 있습니다. 백엔드나 DX 직군을 목표로 한다면 Pandas와 SQL을 함께 익혀 두면 강점이 됩니다.

설비 적용

강사가 실습 데이터를 구하는 곳으로 **Kaggle(캐글)**을 소개했습니다. 구글이 운영하는 사이트로 미국 중심의 공공데이터를 CSV로 내려받을 수 있습니다. 과정이 끝난 뒤 취업 준비 단계에서 개인 프로젝트 데이터를 구할 때 유용합니다. 다만 **미국 기준 데이터가 많아 단위(화씨·마일 등)를 반드시 확인**해야 합니다.

STEP 2

Series와 DataFrame

Pandas의 두 그릇

Pandas에는 데이터를 담는 그릇이 두 개뿐입니다. 작은 그릇이 Series, 큰 그릇이 DataFrame입니다. 이 둘만 알면 Pandas의 절반은 이해한 셈입니다.

2-1. 설비 데이터가 표로 쌓이는 방식

먼저 우리가 다룰 데이터의 모양을 보자. 행은 한 시점의 측정 기록, 열은 하나의 측정 항목입니다.

12_metro_compressor.csv의 실제 모습

index	측정시각	오일온도	모터전류	가동상태	
0	2020-02-27 06:38:47	51.3	6.04	가동	← 행 = 한 시점 기록
1	2020-02-27 07:28:21	56.8	0.04	정지	
2	2020-02-27 08:17:54	55.7	0.03	정지	
3	2020-02-27 09:07:27	NaN	3.81	가동	← 결측 발생
4	2020-02-27 09:57:01	55.3	0.04	정지	
	↑		↑		
열 = 측정 항목 하나 (세로로 같은 의미의 값)					

행은 시간이 지나며 늘어나고, 열은 측정 항목 수만큼 고정됩니다. 다른 설비 데이터도 구조는 똑같습니다. 그리고 3번 행 오일온도가 NaN — 그 시점에 측정값이 없었다는 뜻입니다.

2-2. Series — 한 줄짜리 데이터

자료형 Series 같은 종류 값이 일렬로 늘어선 한 열

정의 표의 한 열에 해당하는 1차원 데이터. 각 값에 인덱스 번호표가 붙어 있습니다.

```
import pandas as pd

s = pd.Series([51.3, 56.8, 55.7, 55.3])
print(s)
print(type(s).__name__)
```

```
▶ 0    51.3
▶ 1    56.8
▶ 2    55.7
▶ 3    55.3
▶ dtype: float64
▶ Series
```

왜 쓰나 따로 외울 개념이 아니다. **DataFrame**에서 열 하나를 꺼내면 그 결과가 **Series**입니다. 우리가 늘 다루는 표의 한 조각입니다.

응용 출석부에 비유하면 이해가 쉽습니다. **학생 번호(인덱스)에 점수(값)가 적힌 것과** 같습니다. 번호로 원하는 값을 바로 찾을 수 있고, 이름표가 붙어 있어 어느 측정의 값인지 추적됩니다.

```
# 인덱스에 이름을 붙일 수도 있다
s = pd.Series([51.3, 56.8, 55.7],
              index=["06:38", "07:28", "08:17"],
              name="오일온도")

print(s)
print("07:28 값:", s["07:28"])

▶ 06:38    51.3
▶ 07:28    56.8
▶ 08:17    55.7
▶ Name: 오일온도, dtype: float64
▶ 07:28 값: 56.8
```

개념

Series가 여러 개 옆으로 모이면 DataFrame이 됩니다. 그래서 "표의 한 조각"이라고 부릅니다.

2-3. DataFrame — 표 전체를 담는 그릇

자료형 DataFrame 여러 Series가 모인 2차원 표

정의 우리가 아는 엑셀 표 그 자체. CSV를 불러오면 바로 이 형태가 됩니다. 앞으로 df라고 쓰면 거의 항상 이것을 가리킵니다.

```
import pandas as pd

df = pd.DataFrame({
    "측정시각": ["06:38", "07:28", "08:17"],
    "오일온도": [51.3, 56.8, 55.7],
    "가동상태": ["가동", "정지", "정지"],
})
print(df)
print(type(df).__name__, df.shape)
```

```
▶ 측정시각 오일온도 가동상태
▶ 0  06:38  51.3   가동
▶ 1  07:28  56.8   정지
▶ 2  08:17  55.7   정지
▶ DataFrame (3, 3)
```

왜 쓰나 원하는 행·열을 자유롭게 선택할 수 있는 것이 강점입니다. "온도 열만 보기", "100도 넘는 행만 골라내기" 같은 작업이 간단한 코드로 끝납니다.

응용 DataFrame과 Series는 쓸 수 있는 기능이 다릅니다. 열 하나를 꺼내면 Series가 되고, 그때부터 `.mean()` 같은 통계 메서드를 바로 쓸 수 있습니다. 이 구분은 다음 회차(18일차)에서 대괄호 겹수로 본격적으로 다룹니다.

```
# 열 하나를 꺼내면 Series
col = df["오일온도"]
print(type(col).__name__, ".", col.mean().round(1))

# 표 전체는 DataFrame
print(type(df).__name__, ".", df.shape)
```

```
▶ Series · 54.6
▶ DataFrame · (3, 3)
```

개념

실무에서는 `pd.DataFrame()`으로 직접 만드는 일보다 `pd.read_csv()`로 파일에서 읽어 오는 일이 훨씬 많습니다. 다음 STEP의 주제입니다.

2-4. 행 · 열 · 인덱스 — 세 단어 정리

용어	영어	무엇인가	설비 데이터에서
행	row	가로 한 줄 — 한 건의 기록	그 시각의 모든 센서값
열	column	세로 한 칸 — 하나의 측정 항목	오일온도 센서의 전체 기록

용어	영어	무엇인가	설비 데이터에서
인덱스	index	각 행에 붙은 번호표	엑셀 왼쪽 행 번호와 같은 역할

주의

인덱스는 0부터 시작합니다. 엑셀은 1부터지만 Pandas는 0부터입니다. 첫 번째 행의 인덱스가 1이 아니라 0이라는 점이 노베이스가 가장 자주 헷갈리는 지점입니다. 200행짜리 데이터의 마지막 인덱스는 200이 아니라 **199**입니다.

개념

Jupyter 노트북에서는 print 없이 **셀에 변수 이름만 적어도** 표가 깔끔하게 정렬되어 출력됩니다. 행이 많으면 위·아래 5줄만 보이고 가운데는 ...로 줄어드는데, **화면만 축약된 것이지 데이터는 그대로**입니다. 값이 없는 칸은 NaN으로 표시됩니다.

STEP 3

CSV와 read_csv

분석의 관문

3-1. CSV 파일이란

개념 CSV (Comma-Separated Values) 쉼표로 값을 구분한 텍스트 파일

정의 한 줄이 표의 한 행, 줄 안에서 값들이 쉼표로 나뉩니다. 첫 줄은 보통 열 이름(헤더).

12_metro_compressor.csv 를 메모장으로 열면

측정시각, 압축압력, 배출압력, 저장압력, 오일온도, 모터전류, 가동상태 ← 헤더

2020-02-27 06:38:47,9.30,-0.02,9.30,51.3,6.04,가동 ← 데이터 행

2020-02-27 07:28:21,8.55,-0.02,8.55,56.8,0.04,정지

2020-02-27 08:17:54,8.67,-0.02,8.67,55.7,0.03,정지

왜 쓰나 엑셀·메모장·파이썬 어디서나 열리는 **범용 형식**입니다. 설비 시스템·공정 DB·공공데이터가 거의 다 CSV로 나옵니다.

응용 같은 파일도 도구에 따라 다르게 보입니다. 메모장으로 열면 쉼표가 그대로 보이고(CSV의 진짜 속살), 엑셀로 열면 쉼표 기준으로 칸이 나뉘어 표로 보입니다. Pandas가 하는 일은 엑셀과 같습니다 — 쉼표로 칸을 나눠 DataFrame으로 바꿉니다.

주의

문제가 생기면 메모장으로 원본을 확인하는 것이 가장 확실한 습관입니다. 엑셀로 보면 이미 해석된 결과라 무엇이 잘못됐는지 안 보입니다. 강사는 VS Code로 열어도 똑같다고 덧붙였습니다.

3-2. 첫 줄 — import pandas as pd

키워드 import pandas as pd Pandas를 pd라는 별명으로 불러오기

정의 import는 외부 도구를 불러오는 명령, as pd는 pandas를 pd라는 짧은 별명으로 부르겠다는 약속.

형태 import pandas as pd

```
import pandas as pd

# 실행해도 출력은 없다 — 오류 없이 넘어가면 성공
print(pd.__version__)
```

▶ 2.3.3

왜 쓰나 전 세계 분석가가 통일해서 쓰는 별명입니다. 다른 별명을 쓰면 남의 코드를 읽기 어려워집니다.

pandas.read_csv 대신 pd.read_csv로 코드도 훨씬 간결해집니다.

응용 15일차 넘파이에서 배운 import numpy as np와 같은 관례입니다. 데이터 분석 파일의 표준 도입부는 아래 세 줄입니다.

```
import numpy as np          # 배열 계산
import pandas as pd         # 표 데이터
import matplotlib.pyplot as plt # 그래프

# 이 세 줄이 데이터 분석 파일의 표준 시작
```

개념

import 줄은 **파일 맨 위에 한 번만** 적습니다. 한 번 불러오면 그 파일 안에서 계속 쓸 수 있습니다.

3-3. pd.read_csv — CSV를 DataFrame으로

함수 pd.read_csv() CSV 파일을 DataFrame으로 읽기

정의 괄호 안에 파일 경로를 따옴표로 감싸 넣으면 Pandas가 읽어 DataFrame으로 돌려줍니다.

형태 df = pd.read_csv("경로/파일이름.csv")

```
import pandas as pd

df = pd.read_csv("data/12_metro_compressor.csv")
print(df.shape)
print(df.head(3))
```

▶ (200, 7)

▶ 측정시각 압축압력 배출압력 저장압력 오일온도 모터전류 가동상태

▶ 0 2020-02-27 06:38:47 9.30 -0.02 9.30 51.3 6.04 가동

▶ 1 2020-02-27 07:28:21 8.55 -0.02 8.55 56.8 0.04 정지

▶ 2 2020-02-27 08:17:54 8.67 -0.02 8.67 55.7 0.03 정지

왜 쓰나 불러오기가 안 되면 그 다음으로 못 넘어갑니다. 사실상 모든 분석의 관문입니다.

응용 CSV만 되는 것이 아니다. `read_excel`(엑셀), `read_json`(JSON), `read_sql`(데이터베이스) 등 형식마다 함수가 준비되어 있습니다. **함수 이름만 바꾸면 되도록 설계되어 있어서**, CSV로 익혀 두면 나머지는 금방입니다.

```
# 파일 형식별 함수 — 이름만 바뀐다
# pd.read_csv("파일.csv")
# pd.read_excel("파일.xlsx")
# pd.read_json("파일.json")

# 읽은 결과는 전부 DataFrame
df = pd.read_csv("data/12_metro_small.csv")
print(type(df).__name__, df.shape)

▶ DataFrame (30, 7)
```

자주 하는 실수

`read_csv` 결과를 변수에 담지 않으면 읽은 데이터가 사라집니다. 다음 항목에서 자세히 다룹니다.

3-4. 결과는 반드시 변수에 — df 관례

강사가 실제로 시연하며 강조한 부분입니다. `pd.read_csv(...)`만 실행하면 읽기는 하지만 **저장이 안 돼 곧바로 사라집니다**. 다음 줄에서 쓸 수 없습니다.

```
답지 않으면 사라집니다
import pandas as pd

# x 변수에 안 담으면
pd.read_csv("data/12_metro_compressor.csv")
# print(df.shape) ← NameError: df is not defined

# ✓ 등호로 담아야 계속 쓸 수 있다
df = pd.read_csv("data/12_metro_compressor.csv")
print(df.shape)

▶ (200, 7)
```

비유

교안의 비유가 좋습니다 — **도서관에서 책을 꺼내 책상 위에 올려두는 일**입니다. 책을 꺼내기만 하면 사라지지만, 책상(변수)에 올려두면 계속 볼 수 있습니다. 등호(=)는 "오른쪽 결과를 왼쪽 이름에 담아라"라는 뜻입니다.

변수 이름	언제	예
df	파일 하나만 다룰 때 — 가장 흔한 관례	DataFrame의 줄임말
df_comp · df_dig	여러 파일을 동시에 다룰 때	의미를 붙여 구분
자유로운 이름	의무는 아님 — 규칙이 아니라 관례	sensor 라고 해도 동작

주의

교안 실습 2 체크포인트에 "변수 이름을 df 대신 sensor로 바꿔도 똑같이 되는지 확인"이라는 항목이 있습니다. 답은 "된다"입니다. df는 규칙이 아니라 모두가 그렇게 쓰는 관례일 뿐입니다. 다만 남이 읽을 코드라면 관례를 따르는 편이 좋습니다.

3-5. 화면에 표를 띄우는 두 가지 방식

DataFrame을 화면에 보여 주는 방법이 환경에 따라 다릅니다. Jupyter 노트북에서는 셀에 변수 이름만 적어도 표가 예쁘게 정렬되어 나오지만, .py 파일에서는 print()로 감싸야 합니다.

환경	코드	결과
Jupyter 노트북	df	HTML 표로 예쁘게 — 테두리·음영까지
.py 파일	df	아무것도 안 나옴
.py 파일	print(df)	글자 표로 출력

출력이 잘려 보일 때

```
# 열이 많아 줄바꿈될 때 — 파일 맨 위에 한 번
pd.set_option("display.width", 200)
pd.set_option("display.max_columns", None)

print(pd.read_csv("data/12_metro_small.csv").head(2))
```

```
▶      측정시각  압축압력  배출압력  저장압력  오일온도  모터전류  가동상태
▶ 0   2020-02-27 06:38:47   9.30  -0.02   9.30   51.3   6.04   가동
```

주의

열이 많으면 터미널 폭에 맞춰 가운데가 ...로 생략되거나 줄바꿈됩니다.

pd.set_option("display.width", 200)과 display.max_columns를 파일 맨 위에 한 번 넣어 두면 해결됩니다. 데이터가 잘린 게 아니라 화면만 축약된 것이므로 계산에는 영향이 없습니다.

STEP 4

경로

read_csv가 안 된다는 분의 90%

교안 p27이 단언합니다 — "read_csv가 안 된다는 분의 90%는 경로 문제". 문법이 틀린 게 아니라 파일이 그 자리에 없는 것입니다. 그래서 경로를 먼저 제대로 잡아야 합니다.

4-1. 상대경로와 절대경로

구분	무엇인가	예	언제
상대경로	지금 작업 폴더 기준 위치	data/12_metro_compressor.csv	보통 이것 — 짧고 이식성 좋음
절대경로	파일의 전체 주소를 처음부터	C:/Users/.../data/...	경로가 완전히 다른 위치일 때

주의

경로는 항상 슬래시(/)로 씁니다. 윈도우 탐색기가 보여 주는 역슬래시(\)를 그대로 붙여 넣으면 파이썬이 이스케이프 문자로 해석해 오류가 납니다. 강사도 "웬만하면 상대경로로 문제를 푼다"고 했습니다.

함수 `os.path.join()` OS에 맞는 경로를 안전하게 만들기

정의 폴더 이름들을 넘기면 운영체제에 맞는 구분자로 이어 붙여 줍니다.

형태 `os.path.join("data", "파일.csv")`

```
import os

path = os.path.join("data", "12_metro_compressor.csv")
print(path)

# 파일이 실제로 있는지 미리 확인
print(os.path.exists(path))

▶ data/12_metro_compressor.csv
▶ True
```

왜 쓰나 윈도우는 \, 리눅스·macOS는 /를 씁니다. `os.path.join`을 쓰면 어느 환경에서 돌려도 경로가 맞습니다. 12일차 모듈·경로 회차에서 배운 내용이 여기서 다시 씁니다.

응용 `os.path.exists()`로 읽기 전에 파일이 있는지 확인하면 오류를 미리 막을 수 있습니다. 실무 코드에서는 이 확인을 넣어 두는 편이 친절합니다.

```
import os, pandas as pd

path = os.path.join("data", "12_metro_compressor.csv")

if os.path.exists(path):
    df = pd.read_csv(path)
    print(f"불러오기 성공: {df.shape}")
else:
    print(f"파일 없음: {path}")
```

▶ 불러오기 성공: (200, 7)

주의

경로 문자열에 폴더 이름을 직접 쓸 때는 **대소문자도 정확히** 맞춰야 합니다. 리눅스·macOS는 Data와 data를 다른 폴더로 봅니다.

4-2. FileNotFoundError — 오류는 겁이 아니라 안내문

가장 흔한 오류

```
import pandas as pd

df = pd.read_csv("12_metro_compressor.csv")    # data/ 를 빠뜨림

# 실행 결과
# FileNotFoundError: [Errno 2] No such file or directory:
#     12_metro_compressor.csv
```

문법 오류가 아니다. 그 경로에 파일이 없다는 뜻이고, 오류 메시지의 **마지막 줄에 핵심**이 적혀 있습니다. 파이썬 오류는 길어 보여도 맨 아래부터 읽으면 대부분 답이 나옵니다.

점검 순서	무엇을 보나	흔한 실수
① 경로	data/ 같은 폴더 이름을 빠뜨리지 않았나	"파일.csv" 처럼 폴더 생략
② 철자	파일 이름 오타·대소문자	equipment ↔ equipment
③ 확장자	.csv를 빠뜨리지 않았나	data.csv ↔ data

설비 적용

실무에서는 경로를 상수로 빼 두는 것이 표준입니다. `DATA_DIR = "data"` 처럼 파일 맨 위에 모아 두면, 폴더 구조가 바뀌어도 한 줄만 고치면 됩니다. 여러 파일을 읽는 코드일수록 효과가 큼니다.

실무형 — 경로 상수 + 존재 확인

```
import os, pandas as pd

DATA_DIR = "data"    # 경로를 한 곳에 모아 둔다

def load(name):
    path = os.path.join(DATA_DIR, name)
    if not os.path.exists(path):
        print(f"[경고] 파일 없음: {path}")
        return None
    return pd.read_csv(path)

df1 = load("12_metro_compressor.csv")
df2 = load("없는파일.csv")
print(df1.shape if df1 is not None else "-")
```

- ▶ [경고] 파일 없음: data/없는파일.csv
- ▶ (200, 7)

4-3. 상대경로의 기준점 — 작업 디렉터리

상대경로는 "**지금 작업 폴더 기준**" 이라고 했습니다. 그런데 그 "지금 작업 폴더"가 어디인지 모르면 경로를 맞출 수 없습니다. 같은 코드가 실행 방식에 따라 되기도 하고 안 되기도 하는 이유가 이것입니다.

경로가 안 맞을 때 가장 먼저 칠 세 줄

```
import os
```

```
# 지금 어디에서 실행되고 있나
```

```
print("작업 폴더:", os.getcwd())
```

```
# 그 폴더에 무엇이 있나
```

```
print("폴더 내용:", os.listdir("."))
```

```
# data 폴더 안을 확인
```

```
print("data 안:", sorted(os.listdir("data"))[:3])
```

▶ 작업 폴더: /home/user/project

▶ 폴더 내용: ['data', 'Practice', 'Python']

▶ data 안: ['12_equipment_sensor.csv', '12_metro_compressor.csv',
'12_metro_compressor_semicolon.csv']

실행 방식	작업 폴더가 되는 곳	흔한 증상
VS Code 실행 버튼	보통 열어 둔 프로젝트 폴더	대체로 잘 맞음
터미널에서 python 파일.py	명령을 친 그 폴더	하위 폴더에서 치면 경로가 어긋남
Jupyter 노트북	.ipynb 파일이 있는 폴더	노트북 위치에 따라 달라짐

자주 하는 실수

"어제는 됐는데 오늘은 안 된다" 는 대부분 작업 폴더가 바뀐 것입니다. 터미널에서 cd로 다른 폴더에 들어간 뒤 실행했거나, VS Code에서 다른 폴더를 열었거나. os.getcwd() 한 줄이면 5초 만에 확인됩니다.

STEP 5

read_csv 옵션 다섯 가지

인코딩 · 구분자 · 인덱스 · 행 · 열

경로가 맞아도 데이터가 이상하게 들어올 수 있습니다. 한글이 깨지거나, 모든 값이 한 열에 뭉치거나, 열이 너무 많아 화면이 정신없거나. 그때 쓰는 것이 **옵션**입니다.

5-1. encoding — 한글이 깨질 때

키워드 encoding= 글자를 해석하는 약속 지정

정의 인코딩은 컴퓨터가 글자를 해석하는 약속입니다. 저장 방식과 읽기 방식이 다르면 깨집니다.

형태 pd.read_csv("파일.csv", encoding="utf-8-sig")

```
import pandas as pd

# 1순위 — utf-8 계열
df = pd.read_csv("data/12_metro_compressor.csv", encoding="utf-8")
print(df.columns.tolist())

# 안 되면 2순위 — cp949 (윈도우 엑셀 저장 파일)
# df = pd.read_csv("파일.csv", encoding="cp949")

▶ ['측정시각', '압축압력', '배출압력', '저장압력', '오일온도', '모터전류', '가동상태']
```

왜 쓰나 한글 열 이름이 영뿔 처럼 깨져 나오면 인코딩이 안 맞은 것입니다. 값이 아니라 **글자 해석의 문제**라 데이터 자체는 멀쩡합니다.

응용 어떤 인코딩인지 **미리 알기 어렵습니다**. 그래서 현실적인 방법은 하나씩 시도해 맞는 것을 찾는 것입니다. 순서는 utf-8 → utf-8-sig → cp949입니다. utf-8-sig의 sig는 파일 맨 앞에 붙는 BOM 표시를 처리해 준다는 뜻입니다.

```
# 인코딩을 하나씩 시도하는 함수
def try_read(path):
    for enc in ["utf-8", "utf-8-sig", "cp949"]:
        try:
            df = pd.read_csv(path, encoding=enc)
            print(f"성공: encoding={enc}")
            return df
        except UnicodeDecodeError:
            print(f"실패: encoding={enc}")
    return None
```

```
df = try_read("data/12_equipment_sensor.csv")
print(df.columns.tolist()[:3])
```

```
▶ 성공: encoding=utf-8
▶ ['설비ID', '측정시각', '온도']
```

주의

utf-8-sig의 sig는 파일 맨 앞에 붙는 **BOM**(Byte Order Mark)이라는 보이지 않는 표시를 처리한다는 뜻입니다. 엑셀에서 저장한 CSV에 자주 붙는데, 이게 남아 있으면 첫 열 이름이 설비ID가 아니라 \ufeff설비ID로 읽혀 KeyError가 납니다. 최신 Pandas는 대부분 알아서 처리하지만, 열 이름이 이상하면 utf-8-sig를 먼저 의심하자.

5-2. sep — 칸이 뭉칠 때

키워드 sep= 값을 나누는 기호 지정

정의 separator의 줄임말. CSV가 항상 쉼표만 쓰는 것은 아니다 — 세미콜론·탭 파일도 흔합니다.

형태 pd.read_csv("파일.csv", sep=";")

```
import pandas as pd

# x 세미콜론 파일을 그냥 읽으면
bad = pd.read_csv("data/12_metro_compressor_semicolon.csv")
print("sep 없이:", bad.shape)

# ✓ sep 을 맞추면
good = pd.read_csv("data/12_metro_compressor_semicolon.csv", sep=";")
print("sep=';' :", good.shape)
```

```
▶ sep 없이: (200, 1)
▶ sep=';' : (200, 7)
```

왜 쓰나 모든 값이 한 열에 뭉쳐 나오면 거의 확실히 구분자 문제입니다. shape이 (200, 1)처럼 열이 1이면 이것을 의심합니다.

응용 증상 → 확인 → 해결의 순서가 정해져 있습니다. ① shape으로 열이 1개인 것을 확인 ② 메모장(또는 VS Code)으로 열어 무슨 기호로 나뉘었는지 확인 ③ 그 기호를 sep에 지정. 자주 쓰는 구분자는 세미콜론(;)과 탭(\t)입니다.

```
# 첫 줄만 읽어 구분자 추측하기
with open("data/12_metro_compressor_semicolon.csv", encoding="utf-8") as f:
    first = f.readline().strip()

print(first[:60])
for mark, name in [(",", "쉼표"), (";", "세미콜론"), ("\\t", "탭")]:
    print(f" {name}: {first.count(mark)}개")
```

- ▶ 측정시각;압축압력;배출압력;저장압력;오일온도;모터전류;가동상태
- ▶ 쉼표: 0개
- ▶ 세미콜론: 6개
- ▶ 탭: 0개

개념

세미콜론이 6개면 열이 7개라는 뜻입니다. 구분자 개수 + 1 = 열 개수.

5-3. index_col · nrow · usecols

키워드 index_col= 특정 열을 행 이름표로

정의 기본 인덱스(0, 1, 2 ...) 대신 지정한 열을 인덱스로 삼습니다.

형태 pd.read_csv("파일.csv", index_col="측정시각")

```
df = pd.read_csv("data/12_metro_small.csv", index_col="측정시각")
print(df.shape)
print(df.head(2))
```

- ▶ (30, 6)
- ▶ 압축압력 배출압력 저장압력 오일온도 모터전류 가동상태
- ▶ 측정시각
- ▶ 2020-02-27 06:38:47 9.30 -0.02 9.30 51.3 6.04 가동
- ▶ 2020-02-27 07:28:21 8.55 -0.02 8.55 56.8 0.04 정지

왜 쓰나 시각이나 ID처럼 각 행을 구분하는 값을 인덱스로 두면, 나중에 그 값으로 행을 바로 찾을 수 있습니다. 시계열 데이터에서 날짜를 인덱스로 두는 것이 표준 패턴입니다.

응용 `index_col`을 지정하면 **그 열이 데이터 열에서 빠집니다**. 위 결과의 `shape`이 (30, 7)이 아니라 (30, 6)인 이유입니다. 열 개수가 하나 줄어든 것이 아니라 인덱스로 옮겨 간 것입니다.

```
a = pd.read_csv("data/12_metro_small.csv")
b = pd.read_csv("data/12_metro_small.csv", index_col="측정시각")
print(a.shape, type(a.index).__name__, "→", b.shape, b.index.name)
```

▶ (30, 7) RangeIndex → (30, 6) 측정시각

개념

이 개념은 18일차 `loc`(라벨 기준 선택)에서 결정적으로 쓰입니다. 인덱스가 글자여도 `loc`으로 부를 수 있습니다.

키워드 `nrows=` · `usecols=` 행·열을 골라 읽기

정의 `nrows`는 위에서 몇 줄만, `usecols`는 필요한 열만 읽습니다.

형태 `pd.read_csv("파일.csv", nrows=10, usecols=["측정시각", "오일온도"])`

```
import pandas as pd

full = pd.read_csv("data/12_metro_compressor.csv")
print("전체 :", full.shape)

few_rows = pd.read_csv("data/12_metro_compressor.csv", nrows=20)
print("nrows=20 :", few_rows.shape)

few_cols = pd.read_csv("data/12_metro_compressor.csv",
                        usecols=["측정시각", "오일온도", "모터전류", "가동상태"])
print("usecols 4개:", few_cols.shape)
```

▶ 전체 : (200, 7)
▶ `nrows=20` : (20, 7)
▶ `usecols 4개`: (200, 4)

왜 쓰나 열이 수십 개인 파일을 통째로 불러오면 **화면도 정신없고 메모리도 낭비**됩니다. 처음 보는 큰 파일은 `nrows`로 구조만 빠르게 보고, 본격 분석은 `usecols`로 필요한 열만 가져옵니다.

응용 둘을 함께 쓸 수 있습니다. 그리고 실습 6이 요구하는 것이 바로 `sep + encoding + usecols`를 한 번에 적용하는 것입니다.

```
# 실습 6 - 옵션 종합
df = pd.read_csv(
    "data/12_metro_compressor_semicolon.csv",
    sep=";",
    encoding="utf-8",
    usecols=["측정시각", "오일온도", "모터전류"],
)
print(df.shape)
print(df.head(2))
```

```
▶ (200, 3)
▶      측정시각  오일온도  모터전류
▶ 0   2020-02-27 06:38:47  51.3   6.04
```

자주 하는 실수

nrows는 **일부만 본** 것입니다. 위쪽 20줄이 멀쩡하다고 전체가 멀쩡하다고 착각하면 안 됩니다. 실제 분석은 전체 데이터로 합니다.

5-4. 옵션 다섯 개 한눈에 — 무엇이 shape을 바꾸는가

옵션마다 결과의 **어느 쪽을 바꾸는지**가 다릅니다. 행을 줄이는 것, 열을 줄이는 것, 열을 인덱스로 옮기는 것, 그리고 아무것도 안 바꾸는 것.

옵션	행 수	열 수	비고
encoding=	그대로	그대로	글자 해석만 바뀜
sep=	그대로	늘어남	맞으면 1열 → 7열
index_col=	그대로	하나 줄어듦	열이 인덱스로 이동
nrows=	줄어듦	그대로	위에서 n줄만
usecols=	그대로	줄어듦	고른 열만

옵션이 shape을 어떻게 바꾸는가

```
import pandas as pd
PATH = "data/12_metro_compressor_semicolon.csv"

실험 = [
    ("옵션 없음",      {}),
    ("sep만",          {"sep": ";"}),
    (" + nrows=20",    {"sep": ";", "nrows": 20}),
    (" + usecols 2개", {"sep": ";", "usecols": ["측정시각", "오일온도"]}),
    (" + index_col",   {"sep": ";", "index_col": "측정시각"}),
]

for name, opts in 실험:
    print(f"{name:14s} → {pd.read_csv(PATH, **opts).shape}")
```

- ▶ 옵션 없음 → (200, 1)
- ▶ sep만 → (200, 7)
- ▶ + nrows=20 → (20, 7)
- ▶ + usecols 2개 → (200, 2)
- ▶ + index_col → (200, 6)

개념

index_col만 열 수가 하나 줄어듭니다. 그 열이 사라진 게 아니라 인덱스로 자리를 옮긴 것입니다. 이 차이를 모르면 "왜 열이 하나 없어졌지?" 하고 헤매게 됩니다.

STEP 6

불러오기 점검 루틴

경로 → 인코딩 → 구분자 → 확인

옵션을 다 배웠으니 하나의 **진단 순서**로 묶습니다. 교안이 병원 진료에 비유한 것처럼, 정해진 순서로 점검하면 원인이 빠르게 드러납니다.



단계	증상	대응	옵션
① 경로	<code>FileNotFoundError</code>	절자 · 확장자 · 위치 순으로 확인	—
② 인코딩	한글이 깨져 보임	utf-8 → utf-8-sig → cp949	encoding=
③ 구분자	값이 한 열에 뭉침	메모장으로 원본 확인 후 기호 지정	sep=
④ 확인	—	불러온 직후 head와 shape — 생략 금지	—

점검 루틴을 함수 하나로

import os, pandas as pd

def safe_read(path, **opts):

점검 루틴을 코드로 옮긴 형태

if not os.path.exists(path): # ① 경로

print(f"[1] 경로 없음: {path}")

return None

for enc in ["utf-8", "utf-8-sig", "cp949"]: # ② 인코딩

try:

df = pd.read_csv(path, encoding=enc, **opts)

break

except UnicodeDecodeError:

continue

else:

print("[2] 인코딩 실패")

return None

if df.shape[1] == 1: # ③ 구분자

print(f"[3] 열이 1개 — sep 확인 필요 (현재 {df.shape})")

print(f"[4] 확인 완료: {df.shape}") # ④ 확인

return df

```
df = safe_read("data/12_metro_compressor_semicolon.csv")
df = safe_read("data/12_metro_compressor_semicolon.csv", sep=";")
```

- ▶ [3] 열이 1개 — sep 확인 필요 (현재 (200, 1))
- ▶ [4] 확인 완료: (200, 1)
- ▶ [4] 확인 완료: (200, 7)

설비 적용

실무 CSV 읽기 코드는 거의 항상 이런 방어 로직을 갖습니다. **현장 파일은 매번 인코딩과 구분자가 다릅니다** — 설비 벤더도, 저장한 사람의 OS도 다르기 때문입니다. `safe_read` 같은 함수를 개인 유틸리티에 넣어 두면 프로젝트마다 재사용할 수 있습니다.

주의

교안 p37이 못 박습니다 — **"불러온 직후 head 확인은 생략 금지"**. `shape`이 맞아도 헤더가 데이터로 읽히거나 숫자 열에 단위 문자가 섞이는 경우가 있습니다. 눈으로 3줄만 봐도 대부분 잡힙니다.

STEP 7

미리보기

head와 tail, 그리고 NaN

여기서부터 교안 12_02다. 데이터를 불러왔으면 **거창한 분석으로 바로 가지 않고 눈으로 먼저 살펴봅니다**. 이것을 **탐색 (explore)** 이라고 부릅니다.

주의

이 STEP의 내용은 녹음 공백 구간(14~15시)에 해당합니다. 교안 12_02 p5~p18, 강사 배포 코드 12_02_01_head, 그리고 직접 폰 Practice/12_02_example.py 실습 1·2로 복원했습니다. 내용 자체는 정확하지만 그 시간대의 구두 설명은 빠져 있습니다.

7-1. 탐색 단계 — 세 가지만 확인합니다

확인 항목	무엇을 보나	도구
① 크기	몇 줄, 몇 열짜리 데이터인가	<code>.shape</code>
② 내용	실제 값은 어떻게 생겼는가	<code>.head()</code> · <code>.tail()</code>
③ 이상	빈 칸이나 이상한 값은 없는가	<code>.info()</code> · 눈으로 확인

교안의 비유가 좋습니다 — **요리로 치면 재료가 얼마나 왔는지 펼쳐 보는 일**입니다. 탐색을 건너뛰면 잘못된 데이터로 분석하게 됩니다.

7-2. head — 앞부분 미리보기

메서드 `.head(n)` 위에서 `n`줄 미리보기 (기본 5줄)

정의 데이터를 불러오면 가장 먼저 치는 확인 명령. 수백만 줄이어도 위 5줄만 빠르게 보여 줍니다.

형태 `df.head()` · `df.head(10)`

```
import pandas as pd
```

```
df = pd.read_csv("data/12_metro_compressor.csv")  
print(df.head(3))
```

```
▶      측정시각 압축압력 배출압력 저장압력 오일온도 모터전류 가동상태  
▶ 0  2020-02-27 06:38:47  9.30 -0.02  9.30  51.3  6.04   가동  
▶ 1  2020-02-27 07:28:21  8.55 -0.02  8.55  56.8  0.04   정지  
▶ 2  2020-02-27 08:17:54  8.67 -0.02  8.67  55.7  0.03   정지
```

왜 쓰나 두 가지를 한눈에 봅니다 — ① 열 이름이 제대로 나왔는지, 값이 칸에 맞는지 ② 빈 칸(NaN)이나 이상한 값이 없는지. 앞 5줄만 봐도 결측 신호까지 잡힙니다.

응용 처음 받은 데이터는 head(20) 정도로 넉넉히 봐서 패턴을 잡는 편이 좋습니다. 5줄로는 반복 패턴이 안 보입니다. 그리고 이 파일은 3번 행 오일온도가 NaN이라 head(5)면 결측이 바로 눈에 들어옵니다.

```
# 5줄로는 안 보이던 것이 보인다
```

```
print(df.head(5)[["측정시각", "오일온도", "가동상태"]])
```

```
▶      측정시각 오일온도 가동상태  
▶ 0  2020-02-27 06:38:47  51.3   가동  
▶ 1  2020-02-27 07:28:21  56.8   정지  
▶ 2  2020-02-27 08:17:54  55.7   정지  
▶ 3  2020-02-27 09:07:27   NaN   가동 ← 결측  
▶ 4  2020-02-27 09:57:01  55.3   정지
```

개념

괄호 안 숫자가 볼 줄 수입니다. 데이터보다 큰 수를 넣어도 오류 없이 있는 만큼만 나옵니다 — 200줄 데이터에 head(500)을 쳐도 200줄만 출력됩니다.

7-3. tail — 뒷부분 미리보기

메서드 .tail(n) 뒤에서 n줄 미리보기

정의 데이터가 끝까지 제대로 쌓였는지 확인합니다. 파일 끝이 깨졌거나 엉뚱한 줄이 붙은 경우를 잡습니다.

형태 df.tail() · df.tail(3)

```
print(df.tail(3))
```

```
▶          측정시각 압축압력 배출압력 저장압력 오일온도 모터전류 가동상태
▶ 197  2020-03-06 16:42:06  9.74 -0.02  9.74  73.0  3.77   가동
▶ 198  2020-03-06 17:31:39  9.89 -0.02  9.89  72.5  3.82   가동
▶ 199  2020-03-06 18:21:13  9.87 -0.02  9.87  71.7  3.75   가동
```

왜 쓰나 시간 데이터는 `tail`로 가장 최근 상태를 확인합니다. 그리고 파일이 중간에 끊겼거나 마지막 줄이 깨진 경우를 여기서 잡습니다.

응용 `head`와 `tail`은 짝으로 쓰는 것이 좋은 습관입니다. 교안 표현대로 "처음과 끝이 멀쩡하면 가운데도 대체로 믿을 만하다". 빠른 1차 점검 도구입니다.

```
# 양 끝을 한 번에 확인
print("[처음]"); print(df.head(2)[["측정시각", "오일온도"]])
print("[마지막]"); print(df.tail(2)[["측정시각", "오일온도"]])
print(f"기간: {df['측정시각'].iloc[0][:10]} ~ {df['측정시각'].iloc[-1][:10]}")
```

```
▶ [처음]
▶          측정시각 오일온도
▶ 0  2020-02-27 06:38:47  51.3
▶ 1  2020-02-27 07:28:21  56.8
▶ [마지막]
▶          측정시각 오일온도
▶ 198  2020-03-06 17:31:39  72.5
▶ 199  2020-03-06 18:21:13  71.7
▶ 기간: 2020-02-27 ~ 2020-03-06
```

개념

양 끝을 보니 이 데이터는 **2020-02-27부터 03-06까지 약 8일치**임을 알 수 있습니다. 그리고 오일온도가 51도에서 71도로 올라가 있습니다 — 기간 중 뭔가 변했다는 첫 신호입니다.

7-4. NaN — 오류가 아니라 결측 표시

개념 NaN (Not a Number) 값이 없음을 나타내는 정상 표시

정의 측정값이 없는 자리에 Pandas가 넣는 표시. **오류가 아니다.**


```
# 결측이 어느 열에 몇 개인지
print(df.isna().sum().to_string())
```

```
▶ 측정시각 0   압축압력 0   배출압력 0   저장압력 0
▶ 오일온도 1    ← 결측 1개
▶ 모터전류 0   가동상태 0
```

왜 쓰나 센서 고장·통신 끊김·점검 중일 때 발생합니다. **NaN은 숫자 0과 전혀 다릅니다** — 0은 측정된 값이고 NaN은 측정 자체가 없었다는 뜻입니다.

응용 강사가 뒤쪽 녹음에서 든 예가 인상적입니다. **설비가 네트워크로 데이터를 보내는데 순간적으로 통신이 끊겼다면 무엇을 저장할 것인가?** 에러 코드를 넣으면 그게 설비에서 온 값인지 여기서 만든 값인지 구분이 안 됩니다. 그래서 **의도적으로 NaN을 넣습니다.**

```
# 결측이 "설비 이상"이라는 뜻은 아니다
row = df[df["오일온도"].isna()]
print(row[["측정시각", "오일온도", "모터전류", "가동상태"]])
```

```
▶           측정시각  오일온도  모터전류  가동상태
▶ 3  2020-02-27 09:07:27    NaN    3.81    가동
▶
▶ → 같은 시각 모터전류는 3.81로 정상 기록 — 설비가 아니라 온도 센서·통신 쪽 문제일 가능성
```

설비 적용

강사의 결론이 정확합니다 — **"NaN이 잡혀 있으니 기계가 이상하다고 무조건 생각할 게 아니다."** 기계는 멀쩡한데 통신이나 서버가 문제일 수 있습니다.

주의

지금 단계의 목표는 발견까지입니다. 결측을 어떻게 처리할지(삭제·대체·보간)는 이후 학습에서 다룹니다. 우선 **"이 데이터에 빈 값이 있다"**는 것을 알아채는 것이 첫걸음입니다.

STEP 8

구조 파악 4종

shape · columns · index · dtypes

여기부터 다시 녹음이 있는 구간입니다. 강사가 shape을 설명하며 먼저 한 말이 "넌파이에서 얘기했던 shape하고 똑같죠?" 였습니다. 16일차에 배운 것이 그대로 이어집니다.

8-1. shape — 행·열 크기

속성 .shape (행, 열) 튜플

정의 DataFrame의 크기를 튜플로 돌려줍니다. **앞이 행, 뒤가 열** — Pandas 전체에서 같은 순서입니다.

형태 df.shape

```
import pandas as pd
df = pd.read_csv("data/12_metro_compressor.csv")
```

```
print(df.shape)
print("행:", df.shape[0], "· 열:", df.shape[1])
```

```
▶ (200, 7)
▶ 행: 200 · 열: 7
```

왜 쓰나 데이터가 얼마나 큰지 한 줄로 답하는 도구입니다. 강사 표현대로 "엑셀 파일을 열면 먼저 보는 게 몇 줄이나 되나, 얼마나 넓나"인데 그것과 똑같습니다.

응용 작업 전후로 shape을 찍어 변화를 확인하는 습관이 좋습니다. 열을 골라 읽었을 때, 행을 걸렀을 때 의도한 대로 됐는지 한 줄로 검증됩니다.

```
before = pd.read_csv("data/12_metro_compressor.csv")
after = pd.read_csv("data/12_metro_compressor.csv",
                    usecols=["측정시각", "오일온도", "모터전류"])
```

```
print("전 :", before.shape)
print("후 :", after.shape)
print("줄인 열:", before.shape[1] - after.shape[1], "개")
```

```
▶ 전 : (200, 7)
▶ 후 : (200, 3)
▶ 줄인 열: 4 개
```

자주 하는 실수

속성이라 괄호가 없다. `df.shape()`라고 쓰면 `TypeError: tuple object is not callable`이 난다.
`head()`는 괄호가 있고 `shape`은 없다 — 이 구분이 오늘 계속 나온다.

8-2. columns — 열 이름 목록

속성 `.columns` 모든 열 이름 목록

정의 열 이름을 Index 객체로 돌려줍니다. 데이터를 골라내는 열쇠.

형태 `df.columns` · `df.columns.tolist()`

```
print(df.columns)
print()
print(df.columns.tolist())
```

- ▶ `Index(['측정시각', '압축압력', '배출압력', '저장압력', '오일온도', '모터전류', '가동상태'], dtype='object')`
- ▶ `['측정시각', '압축압력', '배출압력', '저장압력', '오일온도', '모터전류', '가동상태']`

왜 쓰나 철자 한 글자만 달라도 못 찾습니다. `Temperature`나 `온도` 뒤에 공백이 붙은 것처럼 조금만 달라도 `KeyError`입니다. 고르기 전에 정확한 이름을 확인하는 것이 예방책입니다.

응용 열이 수십 개인 데이터에서는 `head`로 봐도 **가운데 열이 생략**되어 안 보입니다. `columns`는 모든 열 이름을 빠짐없이 보여 줍니다. 그리고 `tolist()`를 붙이면 파이썬 리스트가 되어 번호로 접근할 수 있습니다.

```
cols = df.columns.tolist()
print("열 개수:", len(cols))
print("4번 열 :", cols[4])
print("마지막 :", cols[-1])

# shape 의 열 숫자와 일치하는지 검증
print("일치 여부:", len(cols) == df.shape[1])
```

- ▶ 열 개수: 7
- ▶ 4번 열 : 오일온도
- ▶ 마지막 : 가동상태
- ▶ 일치 여부: True

개념

교안 실습 3 체크포인트가 "columns 열 개수가 shape 열 숫자와 일치하는가" 를 묻습니다. 위 마지막 줄이 그 검증입니다.

8-3. index — 행 이름표 범위

속성 `.index` 행에 붙은 번호표의 범위

정의 columns가 열 이름이면 index는 행 이름입니다. 아무것도 지정하지 않으면 0부터 시작하는 자동 번호.

형태 `df.index`

```
print(df.index)
print("첫 행:", df.index[0], "· 마지막 행:", df.index[-1])
```

▶ `RangeIndex(start=0, stop=200, step=1)`
▶ 첫 행: 0 · 마지막 행: 199

왜 쓰나 행을 가리키는 주소입니다. 표를 합치거나 값을 짝지을 때 인덱스를 기준으로 맞춥니다.

응용 `stop=200`인데 마지막 인덱스는 **199**입니다. 컴퓨터는 0부터 세기 때문에 행이 200개여도 마지막 번호는 199다. 파이썬 전체 규칙이라 리스트·문자열에서도 동일합니다.

```
# index_col 을 지정하면 인덱스가 바뀐다
df2 = pd.read_csv("data/12_metro_small.csv", index_col="측정시각")
print(type(df2.index).__name__)
print("인덱스 이름:", df2.index.name)
print(df2.index[:2].tolist())
```

▶ `Index`
▶ 인덱스 이름: 측정시각
▶ `['2020-02-27 06:38:47', '2020-02-27 07:28:21']`

주의

사람은 1부터, 컴퓨터는 0부터 셉니다. "1번 행을 보려고 1을 넣었는데 두 번째 행이 나오는" 함정이 여기서 생깁니다. 다음 회차 `loc·iloc`에서 결정적으로 중요해집니다.

8-4. dtypes — 열별 자료형

속성 `.dtypes` 각 열이 숫자인지 글자인지

정의 열마다 어떤 자료형인지 보여 줍니다. 세 가지만 알면 90%가 해결됩니다.

형태 df.dtypes

```
print(df.dtypes.to_string())
```

```
▶ 측정시각    object
▶ 압축압력    float64
▶ 배출압력    float64
▶ 저장압력    float64
▶ 오일온도    float64
▶ 모터전류    float64
▶ 가동상태    object
```

왜 쓰나 데이터를 불러온 직후 `dtypes` 확인이 계산 오류를 막아 줍니다. 온도가 글자로 읽히면 평균을 못 구하고, ID가 숫자로 읽히면 앞자리 0이 사라집니다.

응용 강사가 짚은 문제 상황 두 가지가 명확합니다. ① **온도가 글자로 읽히면?** 글자끼리는 계산이 안 되니 평균을 못 구합니다. ② **ID가 숫자로 읽히면?** 001의 앞자리 0이 사라져 1이 됩니다 — 우편번호·제품 코드도 같은 문제입니다.

```
# 숫자처럼 보여도 글자인 경우
import pandas as pd
bad = pd.DataFrame({"온도": ["51.3", "56.8", "55.7"]})
print(bad.dtypes.to_string())

# 글자라서 계산이 안 된다
print("더하면:", (bad["온도"] + bad["온도"]).tolist())

# 숫자로 바뀌어야 계산 가능
bad["온도"] = bad["온도"].astype(float)
print("변환 후:", bad.dtypes.to_string(), "· 평균", round(bad["온도"].mean(), 1))

▶ 온도    object
▶ 더하면: ['51.351.3', '56.856.8', '55.755.7']
▶ 변환 후: 온도    float64 · 평균 54.6
```

자주 하는 실수

글자 열끼리 +를 하면 **오류 없이 이어 붙습니다**. 16일차 넘파이의 <U2 함정과 똑같습니다. 오류가 안 나서 더 위험합니다.

8-5. 세 가지 자료형

자료형	무엇	설비 데이터 예	점검 포인트
int64	소수점 없는 정수	압축기 1 · 타워 0	상태 코드 · 개수
float64	소수점 있는 숫자	오일온도 51.3 · 모터전류 6.04	센서 측정값은 대부분 이것
object / str	글자	측정시각 · 가동상태	숫자여야 할 열이 여기 있으면 문제

주의

교안이 든 비유가 좋습니다 — **휘발유 차에 경유를 넣으면 안 되듯**, 숫자 계산에 글자가 들어가면 멈춥니다. dtypes 점검은 **1초면 되지만 안 하면 몇 시간을 잡아먹습니다**. 분석 시작 전에 한 줄로 확인하자.

설비 적용

12_metro_digital.csv를 보면 압축기·타워·저압스위치가 전부 int64입니다. **0과 1만 갖는 디지털 신호**라서 그렇습니다 — 켜짐/꺼짐을 정수로 기록한 것입니다. 반면 압축기 로그는 압력·온도·전류가 전부 float64입니다. **자료형만 봐도 어떤 종류의 데이터인지 짐작할 수 있습니다**.

STEP 9

info와 describe

첫 탐색 4단계의 완성

9-1. info — 구조를 한 화면에

메서드 `.info()` shape · columns · dtypes · 결측을 종합

정의 행 수 · 열 이름 · 자료형 · 결측을 **한 번에** 보여 주는 종합 도구. 건강검진에 해당합니다.

형태 `df.info()`

```
import pandas as pd
df = pd.read_csv("data/12_metro_compressor.csv")
df.info()
```

```
> <class 'pandas.core.frame.DataFrame'>
> RangeIndex: 200 entries, 0 to 199
> Data columns (total 7 columns):
> #   Column   Non-Null Count  Dtype
> ---  -
> 0   측정시각  200 non-null    object
> 1   압축압력  200 non-null    float64
> 2   배출압력  200 non-null    float64
> 3   저장압력  200 non-null    float64
> 4   오일온도  199 non-null    float64
> 5   모터전류  200 non-null    float64
> 6   가동상태  200 non-null    object
> dtypes: float64(5), object(2)
> memory usage: 11.1+ KB
```

왜 쓰나 shape · columns · dtypes를 따로 칠 필요 없이 **한 화면에서 종합**됩니다. 강사 말대로 "head와 info만 쳐도 데이터의 절반은 파악한 셈"입니다.

응용 Non-Null Count가 핵심입니다. 전체가 200인데 오일온도만 199 — 땀샘 하나로 결측 1개가 잡힙니다. head는 우연히 보이는 것이고, info는 **전체에서 정확한 숫자로** 알려 줍니다.

```
# info 를 표로 다시 정리해 보면
total = len(df)
for col in df.columns:
    nn = df[col].notna().sum()
    miss = total - nn
    mark = " ← 결측" if miss else ""
    print(f"{col:6s} {nn:4d} non-null   결측 {miss}{mark}")
```

```
▶ 측정시각  200 non-null   결측 0
▶ 압축압력  200 non-null   결측 0
▶ 배출압력  200 non-null   결측 0
▶ 저장압력  200 non-null   결측 0
▶ 오일온도  199 non-null   결측 1 ← 결측
▶ 모터전류  200 non-null   결측 0
▶ 가동상태  200 non-null   결측 0
```

자주 하는 실수

shape과 달리 info()는 괄호가 필요합니다. 메서드이기 때문입니다. 강사도 이 차이를 짚었습니다 — "shape과 달리 괄호를 써 줘야 된다."

9-2. info 출력 읽는 법 — 다섯 부분

위치	표시	뜻
① 행 수	RangeIndex: 200 entries	200행 — shape의 앞 숫자와 동일
② 열 수	total 7 columns	7열 — shape의 뒤 숫자와 동일
③ 열 이름	Column	세로로 나열
④ 결측	Non-Null Count	전체보다 작으면 결측 있음 — 핵심
⑤ 자료형	Dtype	int64 · float64 · object

9-3. describe — 다음 회차 예고

이 회차 마지막에 describe()가 짧게 소개됐습니다. 강사가 "시간 상황상 구체적인 설명을 못 드린 주제" 라고 했고, 실제로 다음 날(18일차) 오전에 다시 다뤘습니다. 여기서는 무엇을 하는 도구인지만 짚습니다.

메서드 .describe() 숫자 열 통계를 한 번에

정의 숫자 열만 골라 개수 · 평균 · 표준편차 · 최소 · 분위수 · 최대 여덟 가지를 자동 계산합니다.

형태 df.describe()


```
print(df.describe().round(2))
```

```
▶ 압축압력  배출압력  저장압력  오일온도  모터전류
▶ count    200.00    200.00    200.00    199.00    200.00
▶ mean       9.17     -0.02     9.17     63.18     2.06
▶ std        0.58      0.05      0.58      6.25     2.20
▶ min        8.13     -0.03      8.13     50.10     0.03
▶ max       10.22      0.60     10.22     75.00     6.19
▶
▶ (25% · 50% · 75% 행은 생략)
```

왜 쓰나 수만 줄을 안 보고도 **전체 모습을 한 화면에** 파악합니다. 강사 말대로 "통계에 관련된 각종 기능이 따로 있지만, 결국 `describe` 한 번 호출하면 기본적인 건 다 한 번에 나온다".

응용 `count`가 199인 열이 눈에 띕니다 — `info`의 Non-Null Count와 같은 정보**입니다**. `describe` 한 줄로 결측까지 함께 잡힙니다. 자세한 해석(75%와 `max`의 거리, 평균과 중앙값 비교)은 18일차에서 다룹니다.

개념

`describe()`는 **글자 열을 자동으로 뺍니다**. 위 출력에 측정시각과 가동상태가 없는 이유입니다.

9-4. 첫 탐색 4단계 — 오늘의 결론

강사가 수업을 마무리하며 정리한 문장이 이 회차의 결론입니다 — "`read_csv` 이후에는 `head` · `shape` · `info` · `describe` 이 네 가지를 기본적으로 실행해 주는 게 권장된다."



네 줄로 끝나는 첫 탐색

```
df = pd.read_csv("data/12_metro_compressor.csv")
```

```
print(df.head(3)); print(df.shape)
df.info(); print(df.describe().round(2))
```

▶ ① `head` → ② `shape` → ③ `info` → ④ `describe` — 새 데이터를 만나면 항상 이 네 줄

개념

이 네 단계는 **어떤 데이터를 받아도 똑같이 적용**됩니다. 설비 로그든 매출 데이터든 순서가 같습니다. 그래서 이것을 몸에 익혀 두면 처음 보는 데이터 앞에서 막막해지지 않습니다. 18일차에서 `describe` 해석을 배우면 이 흐름이 완성됩니다.

치트시트

오늘 나온 전부 — 한 장 요약

A-1. read_csv 옵션

옵션	뜻	예	언제 쓰나
encoding=	글자 해석 약속	"utf-8" · "cp949"	한글이 깨질 때
sep=	값을 나누는 기호	"," · "\t"	값이 한 열에 뭉칠 때
index_col=	인덱스로 쓸 열	"측정시각"	시각·ID로 행을 부르고 싶을 때
nrows=	위에서 몇 줄만	20	큰 파일 구조만 빠르게 볼 때
usecols=	읽을 열 목록	["측정시각", "오일온도"]	열이 많아 필요한 것만 볼 때

A-2. 탐색 도구

도구	형태	하는 일	기억할 점
.head(n)	메서드	위에서 n줄 (기본 5)	처음엔 head(20)으로 넉넉히
.tail(n)	메서드	뒤에서 n줄	head와 짝으로
.shape	속성	(행, 열)	괄호 없음 · 앞이 행
.columns	속성	열 이름 목록	.tolist()로 리스트화
.index	속성	행 이름표 범위	0부터 — 200행이면 0~199
.dtypes	속성	열별 자료형	int64 · float64 · object
.info()	메서드	구조 + 결측 종합	Non-Null Count가 핵심
.describe()	메서드	숫자 열 8개 통계	글자 열 자동 제외

A-3. 오류별 대응

오류 / 증상	원인	해결
FileNotFoundError	경로·철자·확장자	폴더 이름 · 오타 · .csv 순으로 점검

오류 / 증상	원인	해결
UnicodeDecodeError 또는 한글 깨짐	인코딩 불일치	utf-8 → utf-8-sig → cp949
shape이 (200, 1)	구분자 불일치	메모장으로 확인 후 sep=";"
NameError: df is not defined	변수에 안 담음	df = pd.read_csv(...)
TypeError: tuple not callable	df.shape() 처럼 속성에 괄호	df.shape
열 이름 KeyError	철자·공백·대소문자	df.columns에서 복사
숫자 열이 object	문자 섞임 · 단위 표기	.astype(float) 또는 원본 확인

A-4. 그대로 쓰는 시작 템플릿

복붙해서 쓰는 시작 템플릿

```
import os
import pandas as pd

DATA_DIR = "data"

def load_and_explore(name, **opts):
    # 불러오기 + 첫 탐색 4단계를 한 번에
    path = os.path.join(DATA_DIR, name)
    if not os.path.exists(path):
        print(f"[경고] 파일 없음: {path}")
        return None

    df = pd.read_csv(path, **opts)
    print(f"==== {name} =====")
    print(f"[shape] {df.shape} · [결측] {df.isna().sum().sum()}개")
    print(f"[자료형] {df.dtypes.value_counts().to_dict()}")
    return df

df = load_and_explore("12_metro_compressor.csv")

▶ ===== 12_metro_compressor.csv =====
▶ [shape] (200, 7) · [결측] 1개
▶ [자료형] {dtype('float64'): 5, dtype('O'): 2}
```

오류·함정 사전

x 이렇게 하면 → 무슨 일이 → ✓ 바로잡기

B-1. 오늘 만나기 쉬운 오류 10가지

x 이렇게 하면	무슨 일이 일어나	✓ 바로잡기
<code>pd.read_csv("파일.csv")</code> (폴더 생략)	<code>FileNotFoundError</code>	<code>pd.read_csv("data/파일.csv")</code>
<code>pd.read_csv(...)</code> (변수에 안 담음)	다음 줄에서 <code>NameError</code>	<code>df = pd.read_csv(...)</code>
<code>"C:\\\\Users\\data.csv"</code> 역슬래시	이스케이프로 해석돼 경로가 깨짐	슬래시 / 또는 <code>os.path.join()</code>
한글 열 이름이 깨져 보임	인코딩 불일치	<code>encoding="utf-8-sig"</code> 또는 <code>"cp949"</code>
세미콜론 파일을 그냥 읽음	<code>(200, 1)</code> 모든 값이 한 열에	<code>sep=";"</code>
<code>df.shape()</code>	<code>TypeError: tuple not callable</code>	<code>df.shape</code> 속성은 괄호 없음
<code>df.info</code>	오류 없이 메서드 객체가 찍힘	<code>df.info()</code> 메서드는 괄호
<code>df["온도"]</code> (공백)	<code>KeyError</code> — 보이지 않는 공백	<code>df.columns</code> 에서 복사해 붙이기
<code>usecols=["없는열"]</code>	<code>ValueError: columns expected but not found</code>	<code>df.columns</code> 로 이름 먼저 확인
NaN을 0으로 착각	평균이 잘못 계산됨	0은 측정값, NaN은 측정 없음 — 다릅니다

B-2. 제출 코드 교정 — 실제로 실행해 확인한 것

`Practice/12_01_example.py`와 `Practice/12_02_example.py`를 돌려 보고 정리했습니다. 오류로 멈추는 코드는 없었고, 실습 1~6을 모두 채웠습니다. 아래는 다듬으면 좋을 부분입니다.

① import와 설정 중복

Practice/12_01_example.py

```
# x 제출 코드 — 같은 import 가 파일 안에서 네 번 반복
import pandas as pd
import os

import pandas as pd      # 실습 1에서 또
import pandas as pd      # 실습 4에서 또
import pandas as pd      # 실습 5에서 또

# ✓ 파일 맨 위에 한 번만
import os
import pandas as pd
```

주의

중복 import는 오류가 나지 않습니다. 파이썬이 이미 불러온 모듈은 다시 읽지 않기 때문입니다. 그래서 눈에 안 띄고 계속 쌓입니다. PEP 8은 import를 파일 최상단에 모아 쓰라고 규정합니다. 코드 리뷰에서 가장 먼저 지적당하는 항목이라, 포트폴리오 레포에 남아 있으면 완성도가 낮아 보입니다.

② 실습 6 — 변수 이름 혼동

Practice/12_01_example.py 실습 6

```
# x 제출 코드 — file_path6 을 열려던 자리에 file_path 가 들어갔다
file_path6 = os.path.join("Data", "12_metro_compressor_semicolon.csv")
with open(file_path, "r", encoding="utf-8") as f:    # ← file_path (실습 3 것)
    reader = csv.reader(f)
    print(next(reader))

# ✓ 의도한 파일을 연다
with open(file_path6, "r", encoding="utf-8") as f:
    print(next(csv.reader(f)))
```

▶ [' 측정시각; 압축압력; 배출압력; 저장압력; 오일온도; 모터전류; 가동상태']

제출 코드는 오류 없이 돌아가지만 엉뚱한 파일(심표 구분 12_metro_small.csv)의 헤더를 출력합니다. 세미콜론 파일을 열었어야 구분자가 ;라는 것이 눈에 보입니다. file_path, file_path3, file_path6처럼 번호를 붙여 관리할 때 흔히 생기는 실수입니다.

개념

이런 실수를 막는 방법은 **번호가 아니라 의미로 이름 짓기**입니다. `file_path6` 대신 `semicolon_path`, `file_path3` 대신 `small_path`처럼 쓰면 잘못된 변수를 넣는 순간 눈에 띄니다. 실습마다 별도 함수로 감싸면 아예 이름이 겹치지 않습니다.

③ 경로 대소문자 혼용

Practice/12_01_example.py 실습 1 · 4

```
# x 같은 파일에서 Data 와 data 를 섞어 씀
file_path = os.path.join("Data", "12_metro_small.csv") # 대문자 D
df = pd.read_csv("data/12_metro_compressor.csv")        # 소문자 d

# ✓ 한쪽으로 통일 — 상수로 빼면 확실하다
DATA_DIR = "Data"
path = os.path.join(DATA_DIR, "12_metro_compressor.csv")
```

자주 하는 실수

윈도우에서는 둘 다 동작합니다. 파일 시스템이 대소문자를 구분하지 않기 때문입니다. 그런데 **리눅스·macOS**에서는 `data` 폴더를 못 찾아 `FileNotFoundError`가 납니다. **서버에 올리거나 팀원과 공유하는 순간 터집니다.** 실무에서 자주 겪는 "내 컴퓨터에서는 되는데" 문제의 전형입니다.

④ 잘 한 부분 — 그대로 유지할 것

부분	무엇이 좋은가
실습 5의 <code>try / except / finally</code>	오류를 잡고 <code>finally</code> 에서 올바른 경로로 복구까지 — 교안 요구를 넘어서
실습 3의 주석	<code># usecols=["가동상태"] : ValueError</code> — 실패 경험을 코드에 남김
12_02 실습 4의 서술 답변	"온도가 object면 문자로 저장됐다는 뜻, 수학적 통계를 낼 수 없다" — 정확한 설명
12_02 실습 3의 한 문장 정리	"데이터 120개가 4개 항목으로 구분되고..." — 교안이 요구한 형식 그대로

설비 적용

특히 실습 5의 `finally` 블록이 좋습니다. 오류가 나든 안 나든 "그래서 올바른 경로로는 되는가" 를 끝까지 확인합니다. 실무 코드에서 파일 처리는 거의 항상 이런 방어 구조를 갖습니다. 13일차 예외처리 회차에서 배운 것을 실제로 적용한 사례입니다.

실습 하이라이트

제출한 것 · 다음 시간

C-1. 오늘 나온 실습 전체

실습	교안 요구	제출	한 줄 평
12_01 실습 1	CSV 불러오기 워밍업	제출	<code>shape</code> · <code>head</code> 확인 — 좋음
12_01 실습 2	설비 센서 CSV 불러오기	제출	<code>head(10)</code> 으로 NaN 위치까지 확인
12_01 실습 3	한글·구분자 깨짐 옵션	제출	실패 시도를 주석으로 남김 — 좋음
12_01 실습 4	필요한 열만 골라 불러오기	제출	경로 대소문자 혼용
12_01 실습 5	경로·옵션 오류 고치기	제출	<code>try/except/finally</code> — 요구 이상
12_01 실습 6	<code>read_csv</code> 옵션 종합	제출	변수 혼동 (<code>file_path6</code> → <code>file_path</code>)
12_02 실습 1	<code>head</code> · <code>tail</code> 로 살펴보기	제출	두 파일 모두 확인
12_02 실습 2	<code>head</code> · <code>tail</code> 행 개수 조절	제출	<code>head(500)</code> 까지 시도 — 좋음
12_02 실습 3	구조 파악 3종 도구	제출	한 문장 정리까지 작성
12_02 실습 4	열 이름·자료형 점검	제출	<code>object</code> 문제를 정확히 설명
12_02 실습 5	<code>info</code> 로 데이터 건강검진	제출	<code>print(df.info())</code> — None이 함께 찍힘
12_02 실습 6	<code>describe</code> 로 이상 신호 찾기	제출	근거 서술 보강 여지

C-2. 사소하지만 알아 둘 것 — info()는 None을 돌려줍니다

Practice/12_02_example.py 실행 5

```
# x 제출 코드 — print 로 감싸면 None 이 함께 찍힌다
print(f"df_5.info:\n{df_5.info()}")
```

```
# 출력 끝에 None 이 붙는 이유:
```

```
# info() 는 화면에 직접 출력하고 반환값은 None 이다
```

```
# ✓ 그냥 호출하면 된다
```

```
df_5.info()
```

```
> <class ...>
> RangeIndex: 120 entries, 0 to 119
> ...
> None      ← print 로 감싸서 생긴 것
```

개념

head()·describe()는 결과를 돌려주는 메서드라 print()로 감싸야 보입니다. 반면 info()는 **화면에 직접 출력하고 None을 돌려주는 메서드입니다. 그래서 print(df.info())**라고 쓰면 정보가 나온 뒤 None이 하나 더 찍힙니다. 오류는 아니지만 출력이 지저분해집니다.

메서드	반환값	쓰는 법
.head() · .tail()	DataFrame	print(df.head()) 또는 Jupyter에서 그냥
.describe()	DataFrame	print(df.describe())
.shape · .columns	tuple · Index	print(df.shape)
.info()	None ← 화면에 직접 출력	df.info() print 없이

C-3. 다음 시간 예고 — 18일차 (8/13)

오늘 describe()를 살짝 보고 끝냈습니다. 다음 날 오전에 그것부터 다시 시작해 **통계 해석**을 배우고, 오후에는 **행·열 선택**으로 넘어갑니다.

다음 시간 주제	핵심 표현	오늘과의 연결
describe 해석	75% vs max · 평균 vs 중앙값	오늘 소개만 하고 끝난 부분
첫 탐색 4단계 정리	head → shape → info → describe	오늘 마지막에 강사가 정리한 그것
열 선택	df['형체력'] vs	Series와 DataFrame 구분이 여기서 쓰임

다음 시간 주제	핵심 표현	오늘과의 연결
	<code>df[['형체력', '실린더압력']]</code>	
인덱스와 라벨	라벨 vs 위치 번호	오늘 배운 <code>index</code> · <code>index_col</code> 의 확장
<code>loc</code> 과 <code>iloc</code>	<code>df.loc[행, 열]</code> · <code>df.iloc[행, 열]</code>	행을 고르는 두 가지 방법
조건 필터링 맛보기	<code>df[df['두께'] >= 13]</code>	16일차 넘파이 불리언과 같은 원리

연결

오늘 배운 `index_col`이 다음 회차에서 결정적으로 쓰입니다. 인덱스를 시각이나 ID로 지정해 두면

`df.loc["2020-02-27 06:38:47"]`처럼 **이름으로 행을 부를 수** 있게 됩니다. 오늘은 옵션 하나로 지나갔지만 개념은 그때 완성됩니다.

CLOSING

맺음말

오늘의 성취와 복습 루틴

오늘 넘은 선 — 배열에서 표로

16일차까지는 숫자만 답을 수 있는 배열을 다뤘습니다. 오늘은 시각·문자·숫자가 섞인 진짜 표로 넘어왔습니다. 설비 로그의 본모습이 이것입니다 — 측정시각은 문자, 압력은 실수, 가동상태는 문자.

그리고 오늘 배운 것의 절반은 **문법이 아니라 습관**입니다. 경로를 먼저 점검하고, 불러온 직후 `head`를 찍고, `info`로 결측을 확인하는 것. 강사가 마지막에 정리한 네 줄 — `head · shape · info · describe` — 이 오늘의 결론입니다.

오늘 할 수 있게 된 것	확인 질문	못 하면
왜 엑셀이 아니라 코드인지 설명합니다	설비 1대 하루 몇 줄인가?	STEP 1
Series와 DataFrame을 구분한다	열 하나를 꺼내면 무엇이 되나?	STEP 2
CSV를 DataFrame으로 읽습니다	결과를 변수에 안 담으면?	STEP 3
경로 오류를 스스로 고칩니다	FileNotFoundError 점검 3단계는?	STEP 4
옵션으로 깨진 파일을 살립니다	shape이 (200, 1)이면 무엇을 의심하나?	STEP 5
점검 루틴을 순서대로 적용합니다	경로 다음은 무엇인가?	STEP 6
head · tail로 양 끝을 확인합니다	NaN과 0은 어떻게 다른가?	STEP 7
구조 4종으로 뼈대를 읽습니다	shape에 괄호가 붙나?	STEP 8
info로 결측을 정확히 셉니다	Non-Null Count가 무엇을 알려주나?	STEP 9

복습 루틴 — 35분

이 회차는 **손에 붙여야 하는 것**이 많습니다. 특히 옵션 다섯 가지는 직접 넣었다 빼 보며 shape 변화를 눈으로 확인해야 기억에 남습니다.



옵션이 shape을 어떻게 바꾸는가

② 옵션 실험 — 하나씩 넣었다 빼며 shape 변화 확인

```
import pandas as pd
PATH = "data/12_metro_compressor_semicolon.csv"
```

```
실험 = [
    ("옵션 없음", {}),
    ("sep만", {"sep": ";"}),
    ("sep + nrows", {"sep": ";", "nrows": 20}),
    ("sep + usecols", {"sep": ";", "usecols": ["측정시각", "오일온도"]}),
    ("sep + index_col", {"sep": ";", "index_col": "측정시각"}),
]
```

```
for name, opts in 실험:
    print(f"{name:16s} → {pd.read_csv(PATH, **opts).shape}")
```

```
▶ 옵션 없음      → (200, 1)
▶ sep만          → (200, 7)
▶ sep + nrows     → (20, 7)
▶ sep + usecols   → (200, 2)
▶ sep + index_col → (200, 6)
```

단계	무엇을	왜
① 부록 A 훑기 (5분)	A-1 옵션표와 A-3 오류표를 눈으로 넘깁니다	증상 → 옵션 대응을 머리에 올립니다
② 옵션 실험 (15분)	위 코드를 실행해 shape 변화를 직접 확인	옵션은 눈으로 봐야 붙습니다
③ 4단계 실행 (10분)	압축기·디지털 신호 두 파일에 head·shape·info·describe	같은 도구로 다른 데이터를 진단
④ 부록 B 교정 (5분)	중복 import 정리 · file_path6 수정 · 경로 통일	제출본 완성도를 올립니다

설비 적용

취업 관점에서 오늘 내용은 **데이터 직군 면접의 첫 질문**을 포함합니다. "CSV를 어떻게 읽나요?"에서 시작해 "한글이 깨지면?", "구분자가 다르면?", "결측은 어떻게 확인하나요?"로 이어집니다. 부록 A-4의 load_and_explore를 레포에 올려 두면 **반복 작업을 함수로 묶을 줄 안다**는 신호가 됩니다.

다음 시간에는 통계를 해석하고 원하는 행·열을 골라내는 법을 배웁니다. **필요한 부분만 정확히 꺼내는 것** — 거기서부터 진짜 분석이 시작됩니다.